

ciaaw

Table of Contents

ciaaw 1

NAME

ciaaw - Command line for ciaaw

SYNOPSIS

ciaaw [OPTION...] [ELEMENT...]

DESCRIPTION

ciaaw is a command line interface which provides the atomic weights, the isotopic compositions and the nuclides atomic weights. If no element is provided the full periodic table is displayed.

OPTIONS

--saw, -s

Get the standard atomic weight.

--ice, -i Get the isotopic composition.

--naw, -n

Get the nuclide atomic weight.

--mu, -m

Get the molar masses in g/mol by multiplying the atomic weights by the molar mass constant M_u .

--colnames, -c

Show the headers in the outputs.

--usage, -u

Show usage text and exit.

--help, -h

Show help text and exit.

--verbose, -V

Display additional information when available.

--version, -v

Show version information and exit.

NOTES

You may replace the default options from a file if your first options begin with @file. Initial options will then be read from the "response file" "file.rsp" in the current directory.

If "file" does not exist or cannot be read, then an error occurs and the program stops. Each line of the file is prefixed with "options" and interpreted as a separate argument. The file itself may not contain @file arguments. That is, it is not processed recursively.

For more information on response files see

https://urbanjost.github.io/M_CLI2/set_args.3m_cli2.html

EXAMPLE

Minimal example

```
ciaaw
ciaaw H C B O Zr Nb --saw --ice --naw --colnames
ciaaw H C B O Zr Nb -sinc
```

SEE ALSO

ciaaw(3), **codata(3)**

NAME

ciaaw - library for isotopic abundances and standard atomic weights

LIBRARY

ciaaw (-libciaaw, -lciaaw)

SYNOPSIS

```
use ciaaw
include "ciaaw.h"
import pyciaaw
```

DESCRIPTION

ciaaw is a Fortran library providing the standard and abridged atomic weights, the isotopic abundance and the isotopesâ standard atomic weights:

- o **SAW** Standard Atomic Weights.
- o **ICE** Isotopic Composition of the Element
- o **NAW** Nuclides Atomic Weight.

The data are taken from <http://ciaaw.org>. C API allows usage from C, or can be used as a basis for other wrappers. Python wrapper allows easy usage from Python.

The latest standard atomic weights were released in 2021 by the *ciaaw* (<https://www.ciaaw.org>). All the values for the atomic weights are provided as double precision reals. The standard atomic weights (or relative atomic mass), **A_r(E)**, are extracted from table 1 from Prohaska et al. (IUPAC Technical Report **94**(5):573â600). For the elements that feature an interval for the standard atomic weight, the mean value and the uncertainty are computed using formulas defined in IUPAC Technical Report **93**(5):629â646. The standard atomic weights are a dimensionless quantity and thus they need to be multiplied by the molar mass constant μ in order to get the value in g.mol⁻¹. See codata for physical constants.

The latest isotopic compositions were released in 2013 by the *ciaaw* (<https://www.ciaaw.org>). All the values for the compositions are provided as double precision reals. The isotopic compositions of the element, are extracted from table 1 from Meija et al. (IUPAC Technical Report. **88**(3):293â306).

The latest atomic weights for nuclides were released in 2020 by *ciaaw* <https://www.ciaaw.org> from Huang et al. (AME 2020 **45**(3):030002). All the values for the nuclide atomic weights are provided as double precision reals.

Fortran API:

- o **function get_version()result(fptra)**
Get the version. Deprecated. It will be removed in the next major release. Use **version()** instead.
- o **character(len=:), pointer :: fptra**
!! Fortran pointer to a string indicating the version..
- o **function capi_get_version()bind(c, name='ciaaw_get_version')result(cptra)**
C API.
- o **type(c_ptr) :: cptra**
!! C pointer to a string indicating the version.
- o **function version()result(fptra)**
Get the version.
 - **character(len=:), pointer :: fptra** !! Pointer to a string (=>version).
- o **function capi_version()bind(C,name='ciaaw_version')result(cptra)**
C API - Get the version

- **type(c_ptr) :: cptr** !! C pointer to a string indicating the version.

o subroutine print_periodic_table()
Print periodic table.

o function is_in_pt(z)result(res)
Check if the atomic number z is in the periodic table

o integer(int32), intent(in) :: z
!! Atomic number

o logical :: res
!! True or False

o function get_z_by_symbol(s)result(res)
Get the atomic number z of the element defined by the symbol s.

o character(len=*), intent(in) :: s
!! Element symbol

o integer(int32) :: res
!! >0 if found and -1 if not found.

o function get_saw(s, abridged, uncertainty)result(res)
Get the standard atomic weight for the element s. Deprecated. It will be removed in the next major release. Use **saw()** instead.

o character(len=*), intent(in) :: s
!! Element symbol.

o logical, intent(in), optional :: abridged
!! Set to False if the abridged value is not desired. Default to TRUE.

o logical, intent(in), optional :: uncertainty
!! Set to True if the uncertainty is desired. Default to FALSE.

o real(dp) :: res
!! NaN if the provided element is incorrect or -1 if the element does not have a SAW.

o function capi_get_saw(s, n, abridged, uncertainty)bind(C, name="ciaaw_get_saw")result(res)
C API. Deprecated. It will be removed in the next major release. Use **capi_saw()** instead.

o type(c_ptr), intent(in), value :: s
!! Symbol.

o integer(c_int), intent(in), value :: n
!! Size of the symbol string.

o logical(c_bool), intent(in), value :: abridged
!! Flag for setting if abridged value is desired.

o logical(c_bool), intent(in), value :: uncertainty
!! Flag for setting if the uncertainty is desired instead of the value.

o real(c_double) :: res
!! NaN if the provided element is incorrect or -1 if the element does not have a SAW.

o function saw(s, ab, u)result(res)
Get the standard atomic weight for the element s.

```

o character(len=*), intent(in) :: s
    !! Element symbol.

o logical, intent(in), optional :: ab
    !! Set to False if the abridged value is not desired. Default to TRUE.

o logical, intent(in), optional :: u
    !! Set to True if the uncertainty is desired. Default to FALSE.

o real(dp) :: res
    !! NaN if the provided element is incorrect or -1 if the element does not have a
    SAW.

o function capi_saw(s, n, ab, u)bind(C, name="ciaaw_saw")result(res)
    C API.

o type(c_ptr), intent(in), value :: s
    !! Symbol.

o integer(c_int), intent(in), value :: n
    !! Size of the symbol string.

o logical(c_bool), intent(in), value :: ab
    !! Flag for setting if abridged value is desired.

o logical(c_bool), intent(in), value :: u
    !! Flag for setting if the uncertainty is desired instead of the value.

o real(c_double) :: res
    !! NaN if the provided element is incorrect or -1 if the element does not have a
    SAW.

o function get_ice(s, A, uncertainty)result(res)
    Get the isotopic composition of the element s for the mass number A.  Deprecated. It will
    be removed in the next major release.  Use ice() instead.

o character(len=*), intent(in) :: s
    !! Element symbol.

o integer(int32), intent(in) :: A
    !! Mass number.

o logical, intent(in), optional :: uncertainty
    !! Set to True if the uncertainty is desired. Default to FALSE.

o real(dp) :: res
    !! NaN if the provided element or the mass number A are incorrect or -1 if the
    element does not have an ICE.

o function capi_get_ice(s, n, A, uncertainty)bind(C, name="ciaaw_get_ice")result(res)
    C API.  Deprecated. It will be removed in the next major release.  Use capi_ice() instead.

o type(c_ptr), intent(in), value :: s
    !! Element symbol.

o integer(c_int), intent(in), value :: n
    !! Size of the symbol string.

o integer(c_int), intent(in), value :: A
    !! Mass number.

o logical(c_bool), intent(in), value :: uncertainty
    !! Flag for returning the uncertainty instead of the value. Default to FALSE.

```

```

o real(c_double) :: res
    !! NaN if the provided element or the mass number A are incorrect or -1 if the
    element does not have an ICE.
o function ice(s, A, u)result(res)
    Get the isotopic composition of the element s for the mass number A.
o character(len=*), intent(in) :: s
    !! Element symbol.
o integer(int32), intent(in) :: A
    !! Mass number.
o logical, intent(in), optional :: u
    !! Set to True if the uncertainty is desired. Default to FALSE.
o real(dp) :: res
    !! NaN if the provided element or the mass number A are incorrect or -1 if the
    element does not have an ICE.
o function capi_ice(s, n, A, u)bind(C, name="ciaaw_ice")result(res)
    C API.
o type(c_ptr), intent(in), value :: s
    !! Element symbol.
o integer(c_int), intent(in), value :: n
    !! Size of the symbol string.
o integer(c_int), intent(in), value :: A
    !! Mass number.
o logical(c_bool), intent(in), value :: u
    !! Flag for returning the uncertainty instead of the value. Default to FALSE.
o real(c_double) :: res
    !! NaN if the provided element or the mass number A are incorrect or -1 if the
    element does not have an ICE.
o function get_nice(s)result(res)
    Get the number of isotopes in ICE of the element s. Deprecated. It will be removed in the
    next major release. Use nice() instead.
o character(len=*), intent(in) :: s
    !! Element symbol.
o integer(int32) :: res
    !! >0 if found or -1 if not found.
o function capi_get_nice(s,n)bind(C, name="ciaaw_get_nice")result(res)
    C API. Deprecated. It will be removed in the next major release. Use capi_nice() in-
    stead.
o type(c_ptr), intent(in), value :: s
    !! Element symbol.
o integer(c_int), intent(in), value :: n
    !! Size of the symbol string.
o integer(c_int) :: res
    !! >0 if found or -1 if not found.
o function nice(s)result(res)
    Get the number of isotopes in ICE of the element s.

```

```

o character(len=*), intent(in) :: s
    !! Element symbol.

o integer(int32) :: res
    !! >0 if found or -1 if not found.

o function capi_nice(s,n)bind(C, name="ciaaw_nice")result(res)
    C API.

o type(c_ptr), intent(in), value :: s
    !! Element symbol.

o integer(c_int), intent(in), value :: n
    !! Size of the symbol string.

o integer(c_int) :: res
    !! >0 if found or -1 if not found.

o function get_naw(s, A, uncertainty)result(res)
    Get the atomic weight of the nuclide s for the mass number A. Deprecated. It will be re-
    moved in the next major release. Use naw() instead.

o character(len=*), intent(in) :: s
    !! Element symbol.

o integer(int32), intent(in) :: A
    !! Mass number.

o logical, intent(in), optional :: uncertainty
    !! Flag for returning the uncertainty instead of the value. Default to FALSE.

o real(dp) :: res
    !! NaN if the provided element or A are incorrect or -1 if the element does not
    have an NAW.

o function capi_get_naw(s, n, A, uncertainty)bind(C, name="ciaaw_get_naw")result(res)
    C API. Deprecated. It will be removed in the next major release. Use capi_naw() in-
    stead.

o type(c_ptr), intent(in), value :: s
    !! Element symbol.

o integer(c_int), intent(in), value :: n
    !! Size of the symbol string.

o integer(c_int), intent(in), value :: A
    !! Mass number.

o logical(c_bool), intent(in), value :: uncertainty
    !! Flag for returning the uncertainty instead of the value. Default to FALSE.

o real(c_double) :: res
    !! NaN if the provided element or A are incorrect or -1 if the element does not
    have an NAW.

o function naw(s, A, u)result(res)
    Get the atomic weight of the nuclide s for the mass number A.

o character(len=*), intent(in) :: s
    !! Element symbol.

o integer(int32), intent(in) :: A
    !! Mass number.

```

```

o logical, intent(in), optional :: u
    !! Flag for returning the uncertainty instead of the value. Default to FALSE.

o real(dp) :: res
    !! NaN if the provided element or A are incorrect or -1 if the element does not
    have an NAW.

o function capi_naw(s, n, A, u)bind(C, name="ciaaw_naw")result(res)
    C API.

o type(c_ptr), intent(in), value :: s
    !! Element symbol.

o integer(c_int), intent(in), value :: n
    !! Size of the symbol string.

o integer(c_int), intent(in), value :: A
    !! Mass number.

o logical(c_bool), intent(in), value :: u
    !! Flag for returning the uncertainty instead of the value. Default to FALSE.

o real(c_double) :: res
    !! NaN if the provided element or A are incorrect or -1 if the element does not
    have an NAW.

o function get_nnaw(s)result(res)
    Get the number of nuclides in NAW of the element s. Deprecated. It will be removed in
    the next major release. Use nnaw() instead.

o character(len=*), intent(in) :: s
    !! Element symbol.

o integer(int32) :: res
    !! >0 if found or -1 if not found.

o function capi_get_nnaw(s,n)bind(C, name="ciaaw_get_nnaw")result(res)
    C API. Deprecated. It will be removed in the next major release. Use capi_nnaw() in-
    stead.

o type(c_ptr), intent(in), value :: s
    !! Element symbol.

o integer(c_int), intent(in), value :: n
    !! Size of the symbol string.

o function nnaw(s)result(res)
    Get the number of nuclides in NAW of the element s.

o character(len=*), intent(in) :: s
    !! Element symbol.

o integer(int32) :: res
    !! >0 if found or -1 if not found.

o function capi_nnaw(s,n)bind(C, name="ciaaw_nnaw")result(res)
    C API.

o type(c_ptr), intent(in), value :: s
    !! Element symbol.

o integer(c_int), intent(in), value :: n
    !! Size of the symbol string.

```

C API:

- char* **ciaaw_get_version**(void)
- char* **ciaaw_version**(void)
- double **ciaaw_get_saw**(char *s, int n, bool abridged, bool uncertainty)
- double **ciaaw_saw**(char *s, int n, bool ab, bool u)
- double **ciaaw_get_ice**(char *s, int n, int A, bool uncertainty)
- double **ciaaw_ice**(char *s, int n, int A, bool u)
- int **ciaaw_get_nice**(char *s, int n)
- int **ciaaw_nice**(char *s, int n)
- double **ciaaw_get_naw**(char *s, int n, int A, bool uncertainty)
- double **ciaaw_naw**(char *s, int n, int A, bool u)
- int **ciaaw_get_nnaw**(char *s, int n)
- int **ciaaw_nnaw**(char *s, int n)

Python API:

- **get_saw**(s: str, abridged: bool = True, uncertainty: bool = False) -> float:
- **saw**(s: str, ab: bool = True, u: bool = False) -> float:
- **get_ice**(s: str, A: int, uncertainty: bool = False) -> float:
- **ice**(s: str, A: int, u: bool = False) -> float:
- **get_nice**(s: str) -> int:
- **nice**(s: str) -> int:
- **get_naw**(s: str, A: int, uncertainty: bool = False) -> float:
- **naw**(s: str, A: int, u: bool = False) -> float:
- **get_nnaw**(s: str) -> int:
- **nnaw**(s: str) -> int:

References

- W.J. Huang, Meng Wang, F.G. Kondev, G. Audi, and S. Naimi. The AME 2020 atomic mass evaluation (I). Evaluation of input data, and adjustment procedures*. **45**(3):030002.
- Juris Meija, Tyler B. Coplen, Michael Berglund, Willi A. Brand, Paul De Bièvre, Manfred Gröning, Norman E. Holden, Johanna Irrgeher, Robert D. Loss, Thomas Walczyk, and Thomas Prohaska. Isotopic compositions of the elements 2013 (IUPAC Technical Report). **88**(3):293â306.
- Thomas Prohaska, Johanna Irrgeher, Jacqueline Benefield, John K. Böhlke, Lesley A. Chesson, Tyler B. Coplen, Tipping Ding, Philip J. H. Dunn, Manfred Gröning, Norman E. Holden, Harro A. J. Meijer, Heiko Moossen, Antonio Possolo, Yoshio Takahashi, Jochen Vogl, Thomas Walczyk, Jun Wang, Michael E. Wieser, Shigekazu Yoneda, Xiang-Kun Zhu, and Juris Meija. Standard atomic weights of the elements 2021 (IUPAC Technical Report). **94**(5):573â600.
- M.H. van der Veen, J. Meia, and D. Brynn Hibbert. Interpretation and use of standard atomic weights (IUPAC Technical Report). **93**(5):629â646.

NOTES

To use *ciaaw* within your fpm project, add the following to your fpm.toml file:

```
[dependencies]
iapws = { git="https://github.com/MilanSkocic/ciaaw.git" }
```

dp stands for double precision and it is an alias to real64 from the iso_fortran_env module.

The definitions of the acronyms:

- o **ASAW** Abridged Standard Atomic Weight
- o **SAW** Standard Atomic Weight
- o **ICE** Isotopic Composition of the Element
- o **NAW** Nuclide Atomic Weight

The definitions of the abbreviations:

- o **s** Element
- o **Z** Atomic number
- o **A** Mass number
- o **u** Uncertainty
- o **ab** Abridged
- o **res** Result

EXAMPLES

Example in Fortran

```
! EXAMPLE IN FORTRAN
program example_in_f
use ciaaw, only: saw, ice, naw, nice, nnaw, version
implicit none(type,external)
print *, "version ", version()

print '(A)', '##### CIAAW SAW #####'
print '(A10, F10.5)', 'ASAW H = ', saw("H", ab=.true.)
print '(A10, F10.5)', 'U ASAW H = ', saw("H", u=.true.)
print '(A10, F10.5)', 'SAW H = ', saw("H", ab=.false.)
print '(A10, F10.5)', 'U SAW H = ', saw("H", ab=.false., u=.true.)
print '(A10, F10.5)', 'ASAW T = ', saw("Tc", ab=.true.)

print '(A)', '##### CIAAW ICE #####'
print '(A, I3)', 'N ICE H = ', nice("H")
print '(A, F12.6)', 'ICE H 1 = ', ice("H", A=1)
print '(A, ES23.16)', 'U ICE H 1 = ', ice("H", A=1, u=.true.)
print '(A, F12.6)', 'ICE H 2 = ', ice("H", A=2)
print '(A, ES23.16)', 'U ICE H 2 = ', ice("H", A=2, u=.true.)
print '(A, I3)', 'N ICE Tc = ', nice("Tc")
print '(A, I3)', 'N ICE C = ', nice("C")

print '(A)', '##### CIAAW NAW #####'
print '(A, ES23.16)', 'NAW H 2 = ', naw("H", A=2)
print '(A, ES23.16)', 'U NAW H 2 = ', naw("H", A=2, u=.true.)
print '(A, I3)', 'N NAW Tc = ', nnaw("Tc")

end program example_in_f
```

Example in C

```
/* EXAMPLE IN C */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
```

```

#include <stdbool.h>
#include "ciaaw.h"

int main(void){
printf("%s0, "##### CIAAW VERSION #####");
printf("version %s0, ciaaw_version());

printf("%s0, "##### CIAAW SAW #####");
printf("%s %10.5f0, "ASAW H   = ", ciaaw_saw("H", 1, true, false));
printf("%s %10.5f0, "U ASAW H = ", ciaaw_saw("H", 1, true, true));
printf("%s %10.5f0, "SAW H    = ", ciaaw_saw("H", 1, false, false));
printf("%s %10.5f0, "U SAW H  = ", ciaaw_saw("H", 1, false, true));
printf("%s %10.5f0, "ASAW Tc  = ", ciaaw_saw("Tc", 2, true, false));

printf("%s0, "##### CIAAW ICE #####");
printf("%s %d0,      "N ICE H   = ", ciaaw_nice("H", 1));
printf("%s %12.6f0, "ICE H 1    = ", ciaaw_ice("H", 1, 1, false));
printf("%s %23.16e0, "U ICE H 1  = ", ciaaw_ice("H", 1, 1, true));
printf("%s %12.6f0, "ICE H 2    = ", ciaaw_ice("H", 1, 2, false));
printf("%s %23.16e0, "U ICE H 2  = ", ciaaw_ice("H", 1, 2, true));
printf("%s %d0,      "N ICE Tc  = ", ciaaw_nice("Tc", 2));
printf("%s %d0,      "N ICE C    = ", ciaaw_nice("C", 1));

printf("%s0, "##### CIAAW NAW #####");
printf("%s %23.16f0, "NAW H 2    = ", ciaaw_naw("H", 1, 2, false));
printf("%s %23.16e0, "U NAW H 2  = ", ciaaw_naw("H", 1, 2, true));
printf("%s %d0,      "N NAW Tc   = ", ciaaw_nnaw("Tc", 2));
return EXIT_SUCCESS;
}

```

Example in Python

```

#!/usr/bin/env python
r"""EXAMPLE IN PYTHON"""
import sys
sys.path.insert(0, "../py/src/")
import pyciaaw

print("##### CIAAW VERSION #####")
print("version ", pyciaaw.__version__)

print("##### CIAAW SAW  #####")
print("ASAW H   = ", pyciaaw.saw("H"))
print("U ASAW H = ", pyciaaw.saw("H", u=True))
print("SAW H    = ", pyciaaw.saw("H", ab=False, u=False))
print("U SAW H  = ", pyciaaw.saw("H", ab=False, u=True))
print("ASAW Tc  = ", pyciaaw.saw("Tc"))

print("##### CIAAW ICE  #####")
print("N ICE H   = ", pyciaaw.nice("H"))
print('ICE H 1   = ', pyciaaw.ice("H", A=1))
print('U ICE H 1 = ', pyciaaw.ice("H", A=1, u=True))
print('ICE H 2   = ', pyciaaw.ice("H", A=2))
print('U ICE H 2 = ', pyciaaw.ice("H", A=2, u=True))
print("N ICE Tc  = ", pyciaaw.nice("Tc"))

```

```
print("N ICE C    = ", pyciaaw.nice("C"))

print("##### CIAAW NAW #####")
print('NAW H 2    = ', pyciaaw.naw("H", A=2))
print('U NAW H 2 = ', pyciaaw.naw("H", A=2, u=True))
print("N NAW Tc   = ", pyciaaw.nnaw("Tc"))
```

SEE ALSO**ciaaw(1), gsl(3), codata(3)**